Circle all of the statements below that are true:

a. $log(n) = O(n)$ T
b. $n = O(log\ n)$ F
c. $log^2(n) = O(n)$ T
d. $n = O(log^2 n)$ F
e. $log(n) = \Omega(n)$ F
f. $n = \Omega(log\ n)$ T
g. $5n^3 + 7n + 13 = O(n^5)$ T
h. $5n^3 + 7n + 13 = \Omega(n^3)$ T
i. $5n^3 + 7n + 13 = O(n)$ F
j. $log(n!) = \Theta(n\ log(n))$ T
k. $n^{1/2} = O(log\ n)$ F
l. $2^n = \Omega(n!)$ F

Answer the following:

A. What is the big-Oh run time of Merge Sort on n items? $O(n\ log\ n)$
B. What is the big-Oh run time of Heap Sort on n items? $O(n\ log\ n)$
C. What is the worst-case big-Oh run time of Quick Sort of n items? $O(n^2)$
D. What is the worst-case big-Oh run time of Selection Sort on n items? $O(n^2)$
E. What is the big-Omega lower bound for comparison based sorting of n items? $\Omega(n\ log\ n)$
F. What is the worst case big-Oh run time to insert 1 item into an AVL-tree that contains n Items? $O(log\ n)$
G. What is the worst-case big-Oh run time to insert 1 item into a normal binary search tree that contains n items? $O(n)$
H. What is the worst case big-Oh run time of Binary Search on an n item sorted list? $O(log\ n)$
I. What is the worst case big-Oh run time to insert 1 item into a min-heap? $O(log\ n)$
J. What is the worst case big-Oh run time to remove the minimum value from a min-heap? $O(log\ n)$
K. What is the run time of breadth-first search (in terms of |V| and |E|)? $O(|V| + |E|)$
L. What is the run time of Dijkstra's algorithm (in terms of |V| and |E|) with a minheap Implementation? $O((|V| + |E|)\ log\ |V|)$
M. What is the run time of Bellman-Ford algorithm (in terms of |V| and |E|)? $O(|V||E|)$

universal sorting bound: $\Theta(n\ log\ n)$

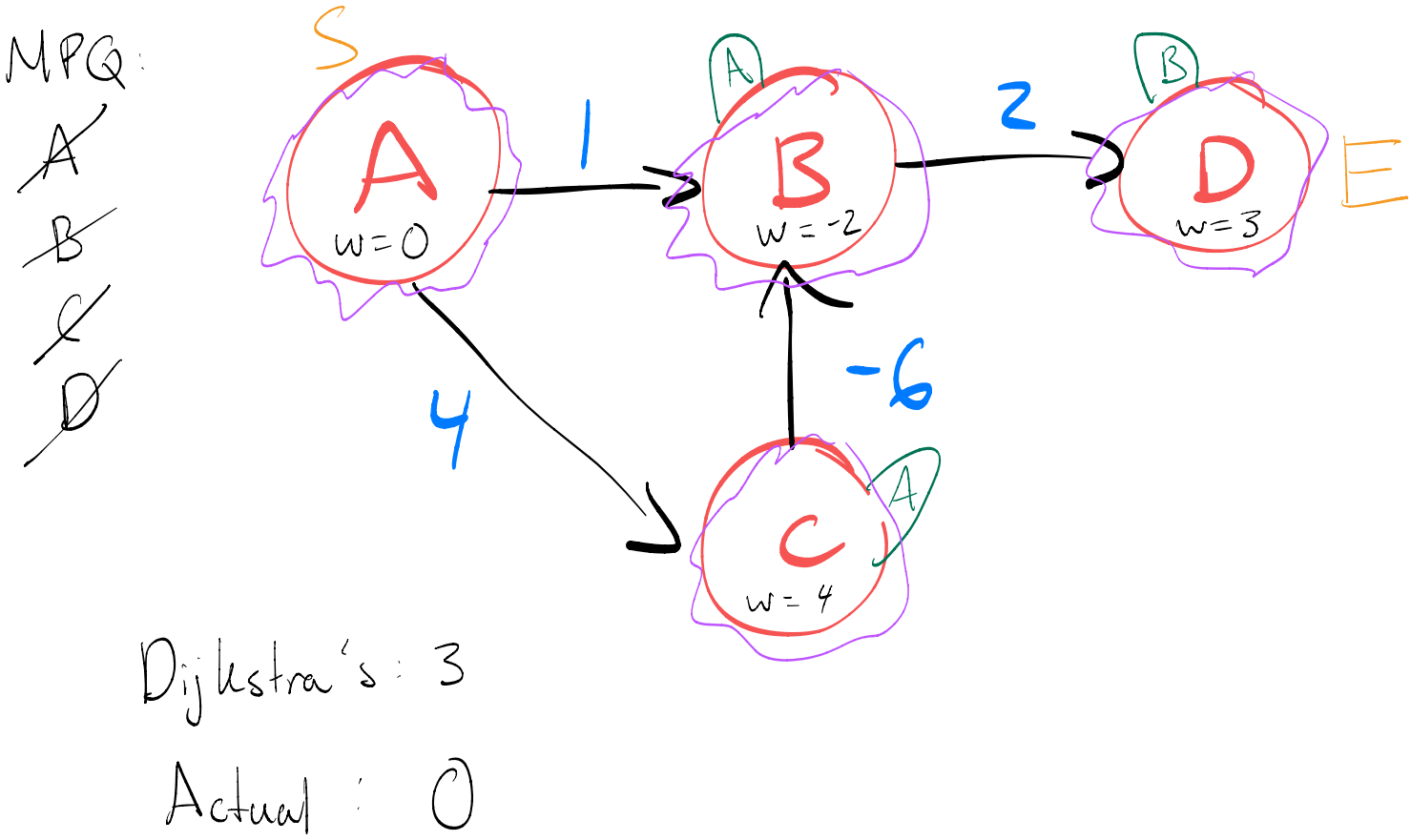
Fill in each blank with either O, Ω, *or* θ, such that the statement is correct and as precise as possible:

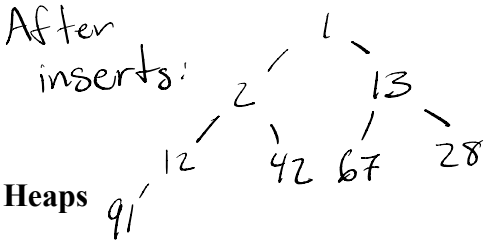
a. $17n^3 + 4n + 5 = O(n^5)$
b. $17n^3 + 4n + 5 = \Theta(n^3)$
c. $17n^3 + 4n + 5 = \Omega(n)$
d. $log(n) = O(n^{1/2})$
e. $n^{1/2} = \Omega(log^2 n)$

Dijkstra's Algorithm

Provide an example where Dijkstra's algorithm fails to find the shortest path in a graph, even when the graph contains no negative weight cycles.
Include:

1) Graph
2) Start and destination vertices for the counterexample
3) The distance computed but Dijkstra's algorithm versus the actual shortest distance





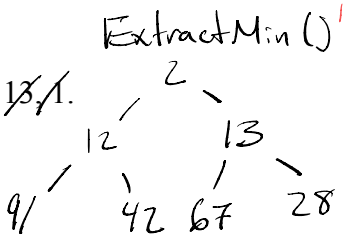
Heaps

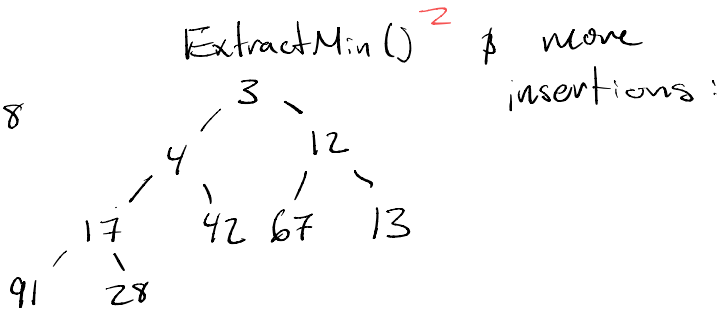
Show the resultant min-heap tree produced by the following sequence of insertions and extractions:

Insert: ~~42~~, ~~28~~, ~~12~~, ~~91~~, ~~2~~, ~~67~~, ~~13~~, 1.

ExtractMin(), ExtractMin(),

Insert: ~~4~~, ~~17~~, ~~3~~.



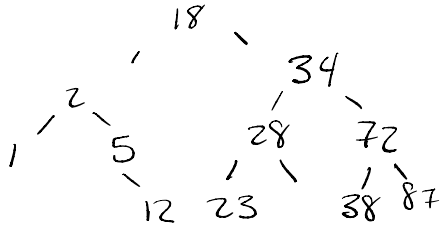


AVL Trees

Show the result of inserting the following keys into an initially empty AVL tree.

To receive partial credit in case of a mistake, show your steps:

~~5~~, ~~18~~, ~~28~~, ~~2~~, ~~34~~, ~~1~~, ~~23~~, ~~12~~, ~~87~~, ~~72~~, ~~38~~.



Hash Tables

Show the result of hashing the following keys to a size 10 hash table using the hash function h(x) = x mod 10 and each of the following 3 different collision resolution schemes:

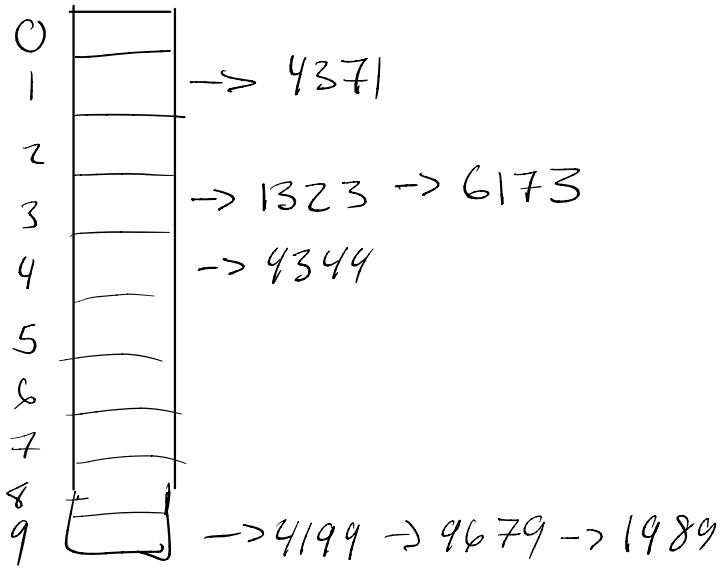
a) Chaining

~~b) Linear probing~~

~~c) Quadratic probing~~

Items to insert:

~~4371~~, ~~1323~~, 6173, ~~4199~~, ~~4344~~, ~~9679~~, ~~1989~~



Heap coding

Implement the void insert(double x) method for a minheap:

class minHeap {

Private:

vector<double> items;

Public:

void insert(double x); // Write this

};

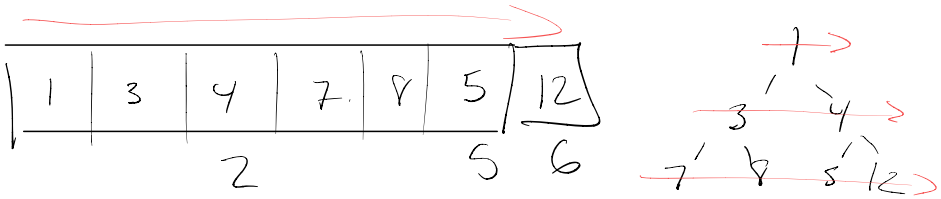
```
void insert(double x)
{
    // Step 1: add to the end of our vector
    items.push_back(x);

    // Step 2: bubble up
    int i = items.size() - 1;
    while (i > 0)
    {
        int parent = (i - 1) / 2;

        // If the heap is valid, stop
        if (items[parent] <= items[i])
            break;

        // Otherwise, heap is still invalid, lets swap with parent
        swap(items[parent], items[i]);

        // Move up the "heap" (vector)
        i = parent;
    }
}
```



(6 - 1) / 2 = 2.5

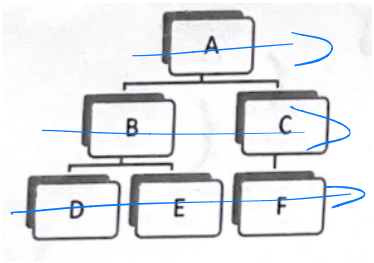
int

Breadth-First Search coding

For the following question, assume binary trees consist of nodes from the following class:

```
class node{
Public:
        int data;
        node * left;
        node * right;
};
```

Write a method levelOrderTraversal(node * r) which prints the items of a binary tree rooted at node r in a "level order", in O(n) time. That is, the first item printed is the root, the next items printed are the children of the root, the next items printed are the grandchildren of the root, etc. You may use containers from the standard template library.
As an example, the following tree would be printed in the following order: A B C D E F




```cpp
void levelOrderTraversal(node* r)
{
    // If tree is empty, do nothing
    if (r == nullptr)
        return;

    // Create a queue, initalized with root (r)
    queue<node*> q;
    q.push(r);

    // Loop until the queue is empty
    while (!q.empty())
    {
        // Get the front-most node
        node* cur = q.front();
        q.pop();

        // Process current node
        cout << cur->data << endl;

        // Add its children to the queue
        // ONLY PUSH CHILD IF NOT NULLPTR
        if (cur->left)
            q.push(cur->left);
        if (cur->right)
            q.push(cur->right);
    }
}
```

Tries coding

Write a function int numEntries(Node * p) that computes and returns the number of strings stored in the Trie rooted at node p. Use the provided Node class.

```
class Node
{
public:
   // Variable denoting if this node is the end
   // character of a string in the Trie
   bool marked;

   Node * children[256];

   Node()
   {
      marked = false;

      for (int i = 0; i < 256; i++)
         children[i] = nullptr;
   }
};
```

```
int numEntries(node* p)
{
    // If Trie is empty, do nothing
    if (p == nullptr)
        return 0;

    // Else create counter variable
    int count = 0;

    // Count ourselves if marked
    if (p->marked)
        count++;

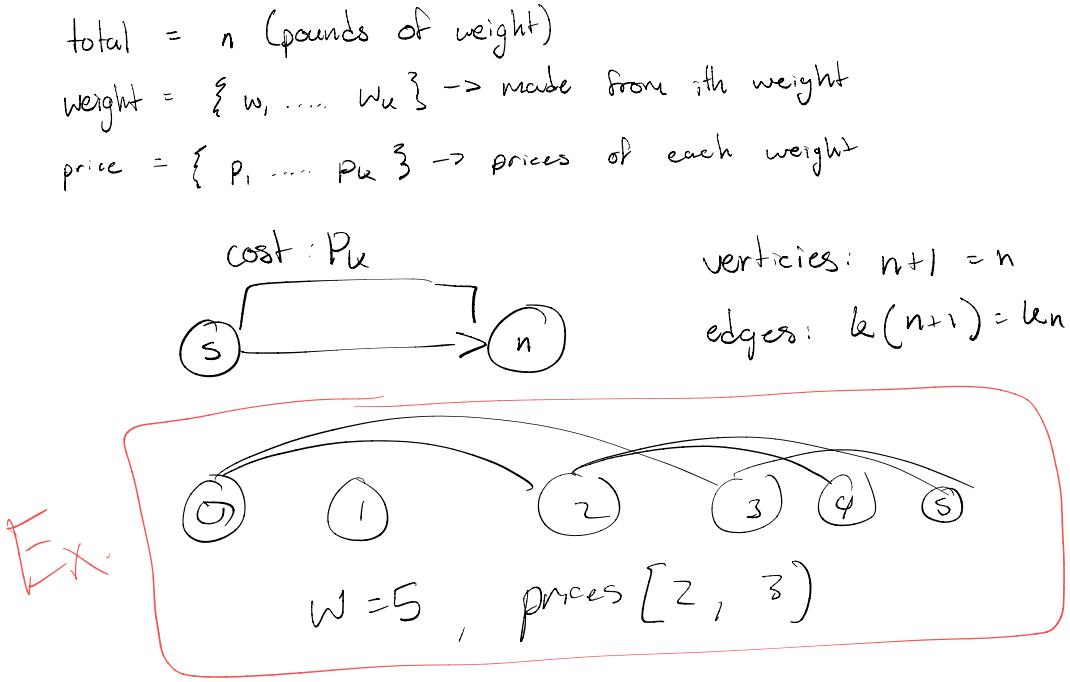
    // Recurse on all children, adding to count as we go
    for (int i = 0; i < 256; i++)
    {
        if (p->children[i])
            count += numEntries(p->children[i]);
    }

    return count;
}
```

Algorithm Design

Suppose you need to purchase several weights that equal exactly n pounds, for a positive integer n (so that you can put them on a barbell and lift it for exercise). The weight store sells k types of weights, where the ith kind of weight has a positive integer weight w_i, and price of p_i dollars, and the store has an unlimited amount of each type of weight for sale. Assume $w_1 = 1$, i.e., they are guaranteed to sell 1 lb weights. Design an algorithm to compute the lowest possible price to purchase exactly n lbs of weight (no more, no less). Describe your algorithm, explain why it will give the cheapest option correctly, and analyze its runtime in terms of n and k.

Hint: Model the problem as a graph problem.




```
// Algo Design

// How to get Pk
    // - Get distance b/w verticies: Wk = n - s
    // - Get associated cost to purchase that weight (distance): Wk->Pk

// Step 1
// Create nodes from S to n (pounds of weight).
// These are the verticies of the graph and s is the starting point

// Step 2
// Connect the nodes (verticies) with directed, weighted edges
    // Check to see if a node plus any Wi values equal a node with value n.
    // If so, add an edge between the current node and the n node

    // Set the cost of that edge to the associated price (Pi) of Wi

// Step 3
// Use Dijkstra's algorithm from starting point s to the shortest path (wegihts and prices)
// to get to n total pounds of weight

// Runtime:
    // O((n + kn)log n) -> O(kn log n)

    // Reminder:
    // Dijkstra's w/ Min Heap is:
        // O((|V|+|E|)log|V|)
```